AI Quiz Generator

A production-oriented TypeScript app: from a README to a grounded quiz and a durable, deterministic result.

The stack

- Node.js 24 · TypeScript · pnpm + Turborepo
- NestJS · Next.js / React
- PostgreSQL + Prisma · Zod
- Vercel AI SDK / OpenAI · Langfuse

The engineering story

- AI creates the questions. Code grades the learner.
- Shared contracts connect the UI and API.
- Source snapshots make results explainable.
- Tests and evals answer different questions.

Eduardo • Project presentation • 11 October 2026

Source: package.json · docs/architecture.md

One language, clear responsibilities

The stack supports end-to-end feature development while keeping the core independent of frameworks.

Product & transport

- Next.js App Router + React: setup, player, results.
- Tailwind + shared shadcn-style primitives.
- NestJS: REST, dependency injection, HTTP errors.

Data & intelligence

- PostgreSQL + Prisma: relational integrity and migrations.
- Zod: validate external inputs and structured output.
- AI SDK + OpenAI: replaceable generation adapter.

Delivery & confidence

- pnpm: workspace dependencies and lockfile.
- Turborepo: task graph, caching and orchestration.
- Jest / RTL / Supertest; Langfuse; GitHub Actions; Docker.

> Frameworks live at the edges; business rules stay testable in plain TypeScript.

Source: apps/api/package.json · apps/web/package.json · turbo.json

Build the reliable path before adding AI

One phase = one specification, a coherent slice, explicit acceptance evidence and a review.

0 → 2 · Foundation

- 0: workspace, conventions and app shells.
- 1: pure domain, scoring and public contracts.
- 2: database + fixture-backed REST lifecycle.

3 → 5 · Interview MVP

- 3: secure Markdown fetch + real generation.
- 4: complete web experience.
- 5: two README demonstrations and reliability.

6 → 8 · Delivery quality

- 6: Langfuse, logs and operational bounds.
- 7: golden dataset and evaluation harness.
- 8: CI, production images and interview docs.

P0: a working assessment flow. P1: observability, quality evidence and a repeatable delivery path.

Source: implementation-roadmap.md · roadmap.md · specs/

Why a monorepo helps AI-assisted development

My choice: give the assistant access to the full feature path, with explicit package boundaries and shared checks.

End-to-end context

- One checkout contains UI, API, contracts, schema, tests and evals.
- An assistant can trace a feature from form to transaction and result.
- A contract change can ship with both consumers.

What Turborepo contributes

- pnpm defines the workspaces; Turbo schedules their tasks.
- Build/typecheck respect dependency order.
- Cached deterministic work shortens feedback loops.
- Database suites run without task caching.

> The benefit is my workflow judgment: visibility comes from the monorepo; Turbo makes its checks practical.

Source: pnpm-workspace.yaml · turbo.json · AGENTS.md

Folders follow the product lifecycle

Context first, role second: a quiz document and an attempt are different concepts.

apps/ · concrete edges

- api/quiz: source, generation, HTTP, persistence, telemetry.
- api/attempt: submission and persistence.
- api/platform: config, logging, HTTP, Prisma.
- web/features/quiz: setup, player, results.

packages/ + evals/ · stable boundaries

- domain: invariants, scoring and port interfaces.
- application: create/get/start/submit/result use cases.
- contracts: public Zod schemas; ui: primitives.
- config: shared tooling; evals/: quality experiments.

> Group code by the reason it changes; avoid a broad infrastructure/ or utils/ dumping ground.

Source: .agents/skills/folder-architecture/SKILL.md · docs/architecture.md

Ports and adapters around a pure core

Runtime calls reach outward through ports; imports point inward toward stable rules.

Inbound

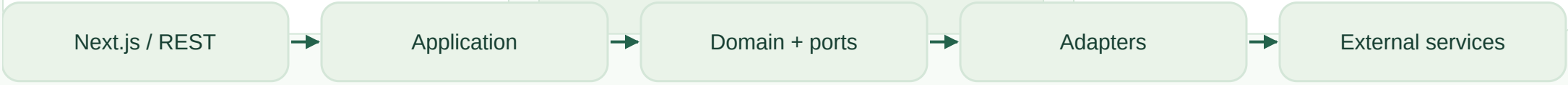
- Browser → public REST
- Nest controllers validate and map HTTP.
- Use cases coordinate the workflow.

Core

- Application use cases
- Domain entities + invariants
- Deterministic scoring
- Port interfaces

Outbound

- MarkdownSourcePort → safe HTTPS
- QuizGenerationPort → AI SDK
- UnitOfWorkPort → Prisma / Postgres
- TelemetryPort → Langfuse or no-op



> Swapping a model, transport or telemetry provider should not rewrite scoring.

Source: packages/domain/src/ · packages/application/src/ · apps/api/src/quiz/quiz.module.ts

Two paths, one durable source of truth

Generation creates an immutable playable quiz; submission records one deterministic result.

Create

- Topic + URL
- → validate and fetch safely
- → normalize + hash + bound context
- → generate + validate + ground
- → atomic quiz persistence

Play & submit

- Public quiz without keys
- → start attempt + private bearer token
- → submit option IDs
- → authorize, validate, grade, finalize
- → stored result with answer review



> The browser never receives a pre-submit answer key and never supplies an authoritative score.

Source: packages/application/src/quiz/create-quiz.ts · packages/application/src/attempt/submit-quiz.ts

Seven tables tell an auditable story

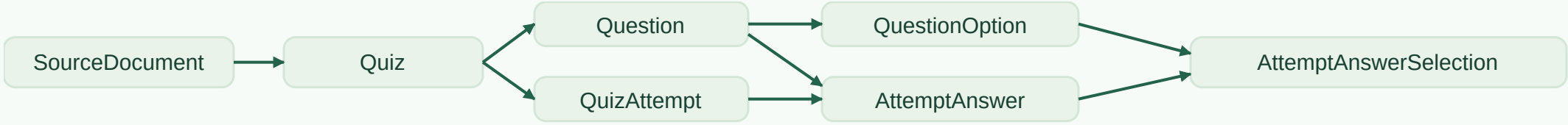
Store the evidence, quiz, key, sitting, selections and calculated result as separate relational records.

Knowledge & quiz

- SourceDocument → Quiz → Question → QuestionOption
- Snapshot, hash, original/resolved URLs, fetchedAt.
- Ordered questions and options.
- isCorrect remains server-only before submission.

Attempt & result

- Quiz → QuizAttempt → AttemptAnswer → Selection
- Token hash, status and timestamps.
- One answer per attempt / question.
- Raw scores + full-precision weighted score.



Snapshot + explicit order + stored selections allow the result to be reconstructed and explained.

Source: apps/api/prisma/schema.prisma · docs/data-model.md

The LLM never grades the learner

The assessment formula is implemented as isolated, deterministic TypeScript policy.

Per question

- Single: correct = 4; wrong = 0.
- Multiple: 1 point per correctly selected option, capped at 4.
- Wrong selections add 0; no deduction.
- Policy is provisional because the task is ambiguous.

Final score

- weight(i) = 1.1 ** i, starting at i = 0.
- score = $\Sigma(\text{score}_i \times \text{weight}_i) / \Sigma(\text{weight}_i)$.
- Store on the 0–4 scale.
- Round score and percentage only for display.

> Consequence: an all-correct multi-select sitting may score below 4; selecting every option is not penalized.

Source: packages/domain/src/scoring/scoring-policy.ts · packages/domain/src/scoring/weighted-score.ts

Every boundary has a different risk

Runtime validation, network restrictions, public projections and transactions protect different parts of the flow.

Remote content & model

- HTTPS / public IP / redirect checks.
- 512 KiB body cap; bounded timeouts.
- Markdown is untrusted source data.
- Structured output + domain validation + excerpt checks.

Browser & persistence

- Public DTOs omit keys and excerpts.
- Random attempt token; only its hash is stored.
- Option ownership checked before grading.
- Atomic, idempotent finalization; sanitized logs.

> Prompt instructions reduce injection risk; they do not prove semantic correctness or complete resistance.

Source: apps/api/src/quiz/source/ · packages/application/src/quiz/to-public-quiz.ts · packages/application/src/attempt/submit-quiz.ts

Four test layers, four kinds of evidence

Fast deterministic checks establish software behavior; live evals measure the uncertain model output.

Unit

- Jest: scoring math, contracts, use cases with fake ports, source boundaries.
- Fast; isolated failures; no paid model needed.

Integration

- Jest + real PostgreSQL: mappings, constraints, rollback, concurrent finalization.
- Proves behavior mocks cannot reproduce.

API E2E & UI

- Jest + Supertest: REST lifecycle with real DB and mocked model.
- Jest + RTL: user interactions.
- Live generation/evals are separate gated runs.

> Supertest is API E2E. RTL is component/UI integration. Neither is a full real-browser E2E suite.

Source: apps/api/test/ · apps/web/src/features/quiz/ · evals/README.md

Evaluate content quality separately

A 14-case golden dataset combines pinned sources, deterministic checks, an independent judge and planned human calibration.

What the harness measures

- Shape, duplicates, excerpt grounding, key leakage.
- Independent key proposal without generator flags.
- Relevance, clarity, ambiguity, distractors, coverage.
- Insufficient-source and adversarial-source cases.

What is evidenced today

- Harness and mocked tests implemented.
- Latest checked-in live report: 14 not_run cases.
- Human calibration: not_run.
- Accepted frozen baseline / regression: not_run.

> A quality score is reviewer evidence. It never overwrites the learner’s 0–4 score.

Source: evals/datasets/golden-v1.json · docs/evals/phase-7-report.md · evals/baselines/

Observe the workflow, compare quality runs

Two integrations: request-stage tracing and offline experiment recording.

Runtime telemetry

- create-quiz → fetch → preprocessing → generation → validation.
- Correlation ID, source hash, strategy, retry and prompt label.
- Token usage on generation; success/error outcome.
- No-op adapter when tracing keys are missing.

Evaluation experiments

- Dataset/rubric versions and per-case outcomes.
- Deterministic + semantic scores and run aggregates.
- Portable Markdown report even without a destination.
- Live dashboard inspection remains gated.

> Tracing is best-effort and redacted; it must not rerun completed business work after an error.

Source: apps/api/src/quiz/telemetry/langfuse-telemetry.adapter.ts · apps/api/src/eval/langfuse-experiment.adapter.ts

Automate confidence; make deployment explicit

Merge-time CI stays deterministic. Credentialed evaluation and release have separate workflows.

Pull request / main CI

- Frozen install → lint → types → Jest.
- Real Postgres → migrations → integration/API E2E.
- Application build → production image builds.
- No model keys; no image push.

Gated paths

- LLM evals: manual + weekly, report artifacts.
- Release: optional SHA-tagged image publishing.
- Multi-stage, non-root API and web containers.
- Deployment script prints operator commands; no host deployed.

> CI configuration exists; hosted green runs, registry publish and an actual deployment require separate evidence.

Source: .github/workflows/ci.yml · .github/workflows/llm-evals.yml · .github/workflows/release.yml

Delivery status with the gaps visible

The repository records core acceptance and local packaging; model-quality and hosted-delivery evidence remain incomplete.

Implemented / documented evidence

- Pure scoring, REST, PostgreSQL and web flow.
- Two live README demonstrations on 11 October.
- Langfuse adapter + limits + health checks.
- Evaluation harness, CI config and local production images.

Remaining work

- Credentialed live eval artifact and Langfuse inspection.
- Human calibration and accepted frozen baseline.
- Registry publish and actual hosted deployment.
- Second generation strategy; real-browser E2E.

> Production-oriented implementation with a repeatable deployment path; the deck does not claim a hosted service.

Source: docs/delivery-status.md · README.md · implementation-roadmap.md

Demo the behavior, then follow the questions

A short opening should lead naturally into a code walkthrough and collaborative implementation.

Demo route

- Express README + routing → generate 5 questions.
- Show four options, type and progress.
- Submit → weighted result + answer review.
- Repeat with Fastify README + plugins.

Code route

- CreateQuizUseCase → source adapter → generation.
- Public projection → submit → scoring policy.
- Prisma schema + transaction + row lock.
- Test → evaluation report → telemetry adapter.

> Historical all-correct scores: Express 3.12 / 4; Fastify 3.60 / 4. Future live questions and scores may differ.

Source: README.md#two-source-demonstration-interview-mvp · docs/walkthrough.md